高级语言程序设计实验报告

南开大学计算机大类

姓名: 李（王景）烨
学号: 2513351
班级: 软件四班
日期: 2026 年 5 月 13 日

目录

1 作业题目 **2**

2 开发软件 **2**

3 课题要求 **2**

4 主要流程 **2**
4.1 整体流程 2
4.2 算法或公式 3
4.2.1 碰撞检测算法 3
4.2.2 敌人生成概率（玩家等级 1 时） 3
4.2.3 玩家升级系统 4
4.2.4 玩家大小表 4
4.2.5 胜利条件 4
4.3 游戏状态机 4

5 单元测试 **4**
5.1 测试案例定义 4
5.2 测试结果 5

6 收获 **5**
6.1 面向对象设计模式 5
6.2 Qt 多媒体编程 5
6.3 游戏性能优化 5
6.4 换皮改造经验 5
6.5 调试与测试技巧 6

1 作业题目

派大星吃汉堡游戏开发（大鱼吃小鱼换皮项目）

2 开发软件

软件名称	版本	用途
Qt Creator	6.xx	集成开发环境
MinGW	8.1.0	C++ 编译器
Qt Framework	6.xx	图形界面与多媒体库

3 课题要求

1. **面向对象设计**：使用类继承体系实现游戏实体
2. **游戏核心逻辑**：碰撞检测、敌人生成、玩家升级系统
3. **用户交互**：键盘控制、冲刺加速、暂停/重启功能
4. **多媒体效果**：背景音乐、胜利/失败音效、动态背景装饰
5. **换皮改造**：将原始“大鱼吃小鱼”游戏改造成“派大星吃汉堡”主题

4 主要流程

4.1 整体流程

实现思路：定义抽象基类 `Fish`，派生出 `PlayerFish`（派大星）和 `EnemyFish`（汉堡敌人），通过虚函数实现多态。

类继承关系图：

```
Fish（抽象基类）

  - pos_: QPointF
  - radius_: qreal
  - tier_: int
  + collidesWith()
  + draw()
  + updateFish()
```

```
PlayerFish              EnemyFish
(派大星)                (汉堡敌人)
+ setMoveDir()          + applyTierSteering()
+ setBoostActive()      + speedMultiplier()
```

游戏主循环流程:

```
// 每帧执行
void GameWidget::onTick() {
    1. 更新帧时间
    2. 更新玩家状态 (位置、能量)
    3. 更新敌人状态 (AI移动)
    4. 碰撞检测与处理
    5. 敌人生成与清理
    6. 渲染画面
}
```

4.2 算法或公式

4.2.1 碰撞检测算法

```
bool Fish::collidesWith(const Fish &other) const {
    const qreal dx = pos_.x() - other.pos_.x();
    const qreal dy = pos_.y() - other.pos_.y();
    const qreal dist = qSqrt(dx * dx + dy * dy);
    return dist < (radius_ + other.radius_);
}
```

原理：两圆碰撞检测，当圆心距离小于两半径之和时发生碰撞。

4.2.2 敌人生成概率（玩家等级 1 时）

```
// 汉堡等级0、1、2的生成概率比为 3:3:4
if (playerTier == 1) {
    const int rand = rng->bounded(10);
    if (rand < 3) enemyTier = 0;       // 30% 小汉堡
    else if (rand < 6) enemyTier = 1; // 30% 中汉堡
    else enemyTier = 2;                // 40% 大汉堡
}
```

4.2.3 玩家升级系统

```
// 升级所需积分表
static const int tbl[] = {3, 8, 20, 30, 80, 120};
```

当前等级	升级所需积分
$1 \to 2$	3
$2 \to 3$	8
$3 \to 4$	20
$4 \to 5$	30
$5 \to 6$	80

4.2.4 玩家大小表

```
static const qreal tbl[] = {29.0, 29.0, 45.0, 45.0, 50.0, 60.0};
```

4.2.5 胜利条件

```
if (score_ >= 1200) {
    gameWon_ = true;
    // 播放胜利音效
}
```

4.3 游戏状态机

状态	触发条件	行为
运行中	游戏正常进行	更新位置、检测碰撞
暂停	按 P 键	停止更新，显示暂停界面
胜利	积分 1200	播放胜利音效，显示胜利界面
失败	被高等级汉堡碰撞	播放失败音效，显示游戏结束界面

5 单元测试

5.1 测试案例定义

测试编号	输入条件	预期输出	测试目的
1	玩家等级 1，碰撞等级 0 敌人	玩家吃掉敌人，+1 分	验证低等级碰撞处
2	玩家等级 1，碰撞等级 2 敌人	玩家被吃掉，游戏结束	验证高等级碰撞处
3	玩家积分解锁升级	玩家等级提升，体积增大	验证升级机制
4	积分达到 1200	触发胜利状态	验证胜利条件
5	边界反弹测试	敌人 20% 概率反弹，80% 概率游出	验证边界行为

5.2 测试结果

测试过程中发现边界反弹概率计算有误,原代码使用 `QRandomGenerator::global()->bounded(1` < 20 实现 20% 反弹概率。经过验证，该逻辑正确无误。

6 收获

6.1 面向对象设计模式

通过抽象基类 `Fish` 定义通用接口，子类实现具体行为:

```
// 基类虚函数定义
class Fish {
public:
    virtual void updateFish(qreal dtSec, const QRectF &bounds,
                            const QPointF &playerPos, int playerTier)
                                = 0;
    virtual void draw(QPainter &painter) const;
};
```

6.2 Qt 多媒体编程

学习了 Qt 6 中 `QMediaPlayer` 与 `QAudioOutput` 的配合使用:

```
bgmPlayer_ = new QMediaPlayer(this);
audioOutput_ = new QAudioOutput(this);
bgmPlayer_->setAudioOutput(audioOutput_);
bgmPlayer_->setSource(QUrl(QStringLiteral("qrc:/skins/bgm.mp3")));
audioOutput_->setVolume(0.5);
```

6.3 游戏性能优化

- 使用 `std::unique_ptr` 管理对象生命周期，避免内存泄漏
- 动态调整敌人生成间隔，平衡游戏难度与性能
- 帧时间计算使用高精度计时器，确保游戏流畅运行

6.4 换皮改造经验

通过修改资源文件（图片、音效）实现游戏主题更换，无需改动核心逻辑代码，体现了良好的代码架构设计。

6.5 调试与测试技巧

在调试过程中，学会了如何使用 Qt Creator 的调试工具定位问题，以及如何通过单元测试验证核心功能的正确性。

附录：核心文件列表

文件名称	功能说明
fish.h/cpp	鱼的抽象基类，定义通用接口
playerfish.h/cpp	玩家鱼（派大星）类
enemyfish.h/cpp	敌人鱼（汉堡）类
gamewidget.h/cpp	游戏主界面，管理游戏逻辑
skins.qrc	资源文件，包含图片和音效
main.cpp	程序入口

——报告结束——